Universitat Rovira i Virgili (URV) y Universitat Oberta de Catalunya (UOC)

Master in Computational and Mathematical Engineering

FINAL MASTER PROJECT

Area: YYY

xxx title of the work xxx

xxx subtitle (if it is the case) xxx

Autor: Complete name of the student

Tutor: Complete name of the director

Barcelona, February 17, 2026

Dr./Dra. (name) ....., certifies that the student (name) ..... has elaborated the work under his/her direction and he/she authorizes the presentation of this memory for its evaluation.

Director's signature:

Credits/Copyright

A page with the specification of credits / copyright for the project (either application on one hand and documentation on the other, or unified), as well as the use of brands, products or services of third parties (including source codes). If a person other than the author collaborated in the project, his identity must be made explicit and what he did.

The most usual case is exemplified below, although it can be modified by any other alternative:

FINAL PROJECT SHEET

Title:	Descriptive of the project
Autor:	Name and surname(s)
Tutor:	Name and surname(s)
Date (mm/yyyy):	Date of delivery
Program:	Master in Computational and Mathematical Engineering
Area:	Name of the FMP area
Language:	English
Key words	Maxim 3 key words

Dedicatory / Quote

Brief words of dedicatory or quote.

Acknowledgments

If deemed appropriate, mention the persons, companies or institutions that have contributed to the realization of this project.

Abstract

Text with the synthesis of the project, that is, a text in which the definition of the project / problem addressed, its objectives / methods of resolution, and the results and conclusions are briefly explained (it can not be a list, but a text continuous written in a structured way). If it is necessary to put a reference in this text, it will be annotated at the bottom of the same page. In this section you can use a more literary and colloquial language than for the rest of the document.

In case of not writing the the document in English, it will be necessary to write the Abstract in duplicate. One of the versions must be textbf obligatorily in English. The other version can be written in Catalan or Spanish, in the language used for the rest of the report. The word Abstract will be changed to " textbf Resum " or " textbf Resumen " in the Catalan and Spanish versions, respectively.

Recommended length: 250 words maximum.

How to write a good abstract (in English):

http://www.ece.cmu.edu/ koopman/essays/abstract.html

Keywords: Work keywords separated by commas. For example for this document they could be Model, Guideline, Template, Memory, Final Master Project / Master

Contents

List of Figures

List of Tables

Chapter 1

Introduction and Planning

1.1 Introduction

Presentación general del trabajo (Simulación de riadas).

1.2 Motivation and Justification

Por qué es importante este trabajo.

1.3 Objectives

Objetivos generales y específicos.

1.4 Project Planning and Management

The successful execution of the Final Master's Project (FMP), which focuses on developing a scientific C++ program for flood simulation, requires robust planning and diligent management. This course is weighted at 18 ECTS, equating to an estimated dedicated effort of approximately 450 hours. The timeline presented below serves as a guiding roadmap, designed to track progress against key project milestones and deliverables.

1.4.1 Work Methodology

The project methodology is structured around the four mandatory phases defined for the FMP, integrated with a phased software development approach. This hybrid methodology ensures that the core theoretical and design elements (Phases 1 & 2) are completed before critical implementation and validation (Phase 3) and final documentation (Phase 4). The iterative nature of the plan allows for necessary adjustments and includes dedicated time for thorough testing and writing throughout the academic year.

1.4.2 Project Schedule: Gantt Chart

The following Gantt chart (Figure **??**) illustrates the proposed 24-week timeline for the project, distributing the estimated 450 working hours across the defined phases. Time buffers have been implicitly integrated into the task durations to account for unforeseen issues, necessary code refactoring, and academic holidays.

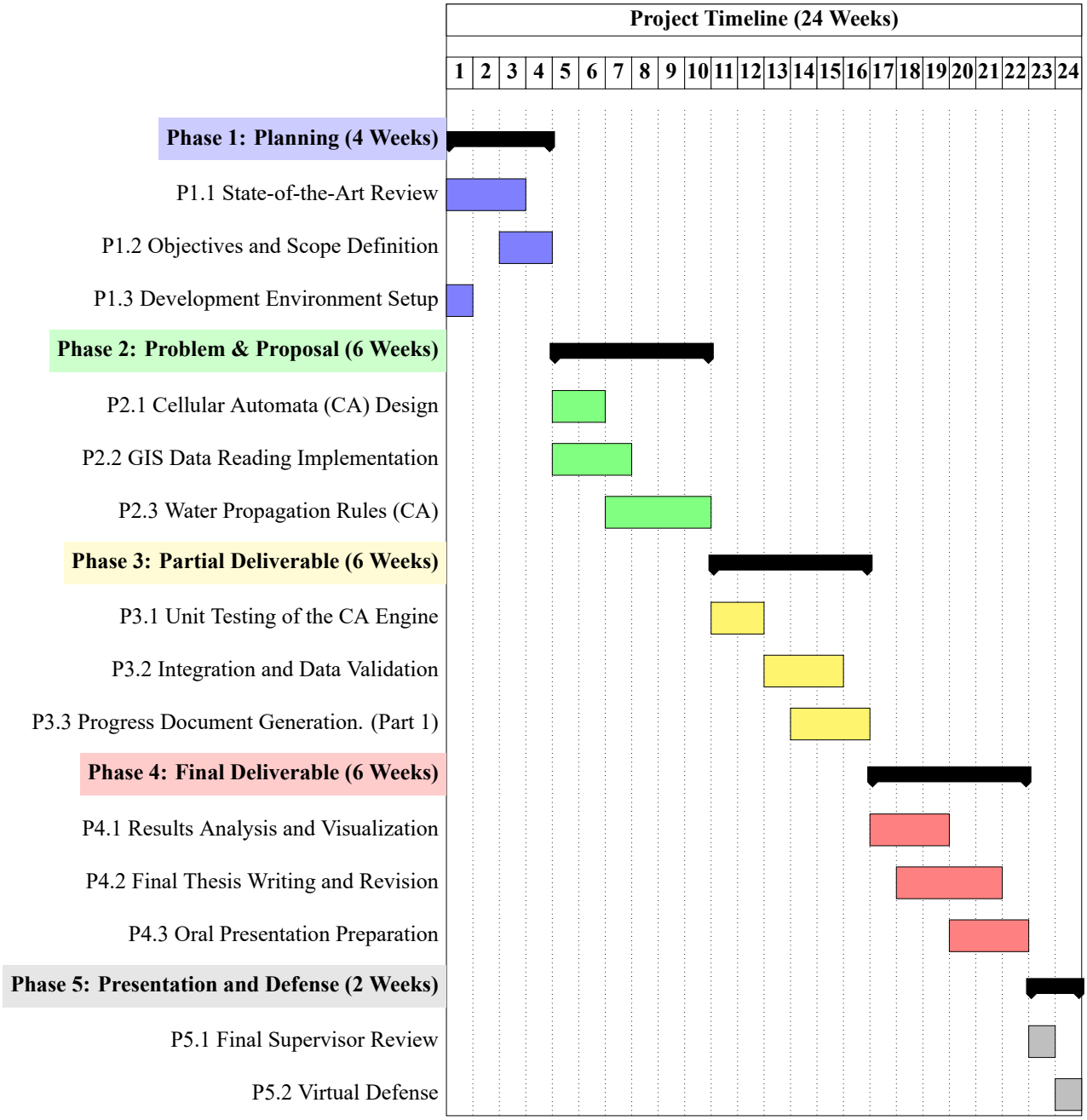

Figure 1.1: Project Timeline (24 Weeks)

1.4.3 Details of Activities

The following table provides a detailed breakdown of the tasks defined in the timeline, specifying the scope of work for each activity.

Table 1.1: Detailed Breakdown of Final Master's Project Activities.

ID	Activity	FMP Phase	Description (2-5 Lines)
PHASE 1: PLANNING			
P1.1	State-of-the-Art Review	Phase 1	Review existing scientific literature on flood simulation, focusing on Cellular Automata (CA) and C++ flow models. Key CA models suitable for water physics will be identified and the core references for the thesis will be collected.
P1.2	Objectives and Scope Definition	Phase 1	Clearly specify the flood scenarios the program will simulate (e.g., 2D vs. 3D, terrain type). Success criteria will be established, and the exact GIS data format required for input topography will be defined.
P1.3	Development Environment Setup	Phase 1	Install and configure the C++ compiler (including necessary GIS libraries like GDAL), the version control system (Git), and development tools. A basic LaTeX project structure will be created to initiate the project documentation.
PHASE 2: PROBLEM DESCRIPTION AND PROPOSAL			
P2.1	Cellular Automata (CA) Design	Phase 2	Define the transition rules and neighborhood structure for the CA (e.g., Moore or von Neumann). The C++ data structure will be designed to efficiently represent terrain cells and water depth.
P2.2	GIS Data Reading Implementation	Phase 2	Develop the C++ module for correctly reading and parsing the input GIS data (Digital Elevation Model or DEM). This module must initialize the initial state of the Cellular Automata based on the topography and the flood starting point.

Table 1.1: Detailed Breakdown of Final Master's Project Activities.

ID	Activity	FMP Phase	Description (2-5 Lines)
P2.3	Water Propagation Rules (CA)	Phase 2	Code the core of the simulator, which includes the equations and logic for water transfer between cells in each iteration. This represents the underlying flow model that governs the flood simulation.
PHASE 3: PARTIAL DELIVERABLE			
P3.1	Unit Testing of the CA Engine	Phase 3	Conduct exhaustive testing of critical code functions, ensuring that GIS data reading, terrain initialization, and, mainly, the CA water propagation rules behave as expected. Focus will be on identifying precision and performance errors.
P3.2	Integration and Data Validation	Phase 3	Run the complete simulator with a representative set of GIS data. Simulation results will be compared against known flood data or other model outputs to validate the program's behavior.
P3.3	Progress Document Generation (Part 1)	Phase 3	Compile and draft the initial chapters of the FMP document in LaTeX. This includes the introduction, literature review, and a detailed description of the development methodology and the Cellular Automata design.
PHASE 4: FINAL DELIVERABLE			
P4.1	Results Analysis and Visualization	Phase 4	Process the simulation output data (depth maps, flooding time, etc.). External tools or C++ libraries will be used to generate clear graphs and visualizations that allow the interpretation of the flood impact.
P4.2	Final Thesis Writing and Revision	Phase 4	Complete the Results and Conclusions chapters of the LaTeX document. A full review of style, grammar, and formatting will be performed to ensure compliance with academic standards.

Table 1.1: Detailed Breakdown of Final Master's Project Activities.

ID	Activity	FMP Phase	Description (2-5 Lines)
P4.3	Oral Presentation Preparation	Phase 4	Design and create the presentation slides. The speech will be rehearsed, emphasizing the justification of the CA model, the C++ development process, and the key results obtained with the GIS data.
PHASE 5: PRESENTATION AND VIRTUAL DEFENSE			
P5.1	Final Supervisor Review	Phase 5	Final session with the FMP supervisor to address last-minute corrections to the document, code, and presentation. Minor details will be adjusted before the final submission and defense.
P5.2	Virtual Defense	Phase 5	Formal presentation of the FMP to the evaluation committee. Technical decisions, methodology, and project conclusions will be defended.

1.5 Document Structure

Breve resumen de los contenidos de los siguientes capítulos.

Chapter 2

State of the Art

Modelos de AC existentes, modelos de flujo de agua, uso de datos GIS.

Chapter 3

Methodology and Design

3.1 Theoretical model: Multilayer Cellular Automata ($m : n - CA^k$)

3.1.1 Fundamentals of the Fonseca, Colls, and Casanovas model applied to fluids

3.1.2 Definition of cellular space and Moore's neighborhood in the context of floods

3.1.3 Formalization of the Global State (EG) and Tensor Transition Functions

3.2 System Architecture Design

3.2.1 The Simulator Core: State inference logic

3.2.2 Hexagonal Architecture: Decoupling Using Ports and Adapters

3.2.3 Advantages of the design for physical layer extensibility

3.3 Data Structure and Tensor Model

3.3.1 Representation of the system as a multi-channel tensor

The core of the simulator is designed around a multi-layered tensor structure that implements the $m : n - CA^k$ formalization. In this model, the global state (EG) of any cell at coordinates (x, y) is determined by the combination of m layers. In our implementation, these layers are represented as channels within a high-performance tensor, allowing the simulation core to perform state transitions using vectorized operations.

3.3.2 Layer Metadata Specification

To formalize the interaction between the Hexagonal Architecture and the tensor core, each layer $\mathcal{L}$ is defined by a set of features $\mathcal{F} = \{R, T, S\}$, where:

Layer Role (R) Defines if the layer is the main state variable or a secondary variable.

- *Main*: The main variable being simulated (k in $m : n - CA^k$).
- *Secondary*: Input data that informs the transition function.

Data Type (T) The primitive type used in the C++ tensor implementation.

- *Float*: Standard for hydraulic variables (depth, slope).
- *Integer*: Used for categorical masks or land-use IDs.

Layer Source (S) Determines the loading and updating logic in the Hexagonal Adapters.

- *Static Raster*: Persistent GIS data loaded at initialization.
- *Temporal Raster*: External sequences (e.g., rainfall) updated during runtime.
- *Derived Layer*: Internally calculated from other layers (e.g., slope from elevation).

Before analyzing the specific definition of each data layer, it is essential to categorize them based on their physical nature and their management within the simulation engine. The following table summarizes the currently defined layers:

Layer	Data type	Role	Source
Elevation	Float	Secondary	Static
Water Depth	Float	Main	Static
Roughness	Float	Secondary	Static
Rainfall	Float	Secondary	Temporal

Table 3.1: Layer Metadata Specification

3.3.3 Definition of layers

In the following, we detail the characteristics and extraction methodology for each of the layers.

3.3.3.1 Elevation Layer

The Elevation Layer is the fundamental topographic descriptor of the simulation environment. It represents the Digital Elevation Model (DEM), providing the vertical coordinate (altitude in meters) for every cell in the cellular automata grid. Within the $m : n - CA^k$ framework, this layer acts as a constant physical constraint that dictates the potential energy available for water movement.

Characteristics

- **Layer Role**: Secondary. It provides environmental context but is not the primary evolving state.
- **Data Type**: Float (32-bit). High precision is required to capture subtle topographic gradients.
- **Layer Source**: Static Raster. The terrain is assumed to be invariant during the flood event.

Extraction Methodology

The extraction process handles the IDRISI32 raster format, which is characterized by a dual-file structure: the data file (.rst) and the documentation/metadata file (.rdc).

3.3.3.2 Water Depth Layer

The Water Depth Layer is the cornerstone of the simulation, acting as the Main Layer ($k = 1$) in the $m : n - CA^k$ formalization. It represents the height of the water column (in meters) above the terrain for each cell. Unlike environmental constraints, this layer constitutes the internal state of the cellular automaton that the Core modifies at every time step (Δt).

Characteristics

- **Layer Role**: Main. It is the main variable under study and the output of the transition function.
- **Data Type**: Float (32-bit). Precision is vital for stability in shallow water equations and low-flow areas.
- **Layer Source**: Static Raster (Initial Conditions). While the layer is dynamic during execution, its starting point is defined by a static input file representing the initial state or ”hot start” conditions.

Extraction Methodology

Since this layer is treated as a Static Raster for initialization purposes, it follows the IDRISI32 extraction pipeline. This allows the simulator to begin from a non-zero state (e.g., simulating a river that already has water or continuing a previous simulation).

3.3.3.3 Roughness Layer

The Roughness Layer provides the spatial distribution of friction coefficients across the simulation grid. It is a critical parameter in the Manning equation, which relates the flow velocity to the hydraulic radius, the slope, and the surface roughness.

Characteristics

- **Layer Role**: Secondary. It acts as a stationary parameter that influences the transition function but is not modified by the simulation.
- **Data Type**: Float (32-bit). Manning coefficients are small decimal values (e.g., 0.013 for concrete, 0.040 for shrubland).
- **Layer Source**: Static Raster. The land use characteristics are assumed to remain constant during the event.

Extraction Methodology

The data for this layer is stored in IDRISI32 format, ensuring compatibility with the other static inputs of the system.

3.3.3.4 Rainfall Layer

The Rainfall Layer defines the net precipitation intensity reaching the surface of each cell. Within the $m : n - CA^k$ framework, it acts as a Secondary Layer that provides a time-dependent input (R) to the transition function. This input is directly integrated into the mass balance equation of the Water Depth layer.

Characteristics

- **Layer Role**: Secondary. It acts as an external forcing or "event" parameter.
- **Data Type**: Float (32-bit). Represented in mm/h.
- **Layer Source**: Time-Varying Raster (Temporal Source). The data is updated at specific intervals during the simulation.

Extraction Methodology

The methodology for populating this layer involves a complex transformation from raw meteorological data to a structured tensor format.

1. **Data Acquisition**: Precipitation data is sourced from sensor networks managed by Hydrographic Confederations (e.g., Automatic Hydrological Information Systems - SAIH). This data typically arrives as point-based time series from rain gauges or as radar-derived grids.

2. **Spatial Interpolation and DEM Alignment**: Since sensor data is often point-based, a spatial interpolation (such as Inverse Distance Weighting or Kriging) is applied to generate a continuous surface. The DEM is used here as a spatial reference to ensure the interpolated rainfall grid matches the simulation's bounding box and resolution. Additionally, topographic features from the DEM can be used to apply orographic corrections to the rainfall distribution.

3. **Temporal Packaging (HDF5)**: The processed grids are stored in HDF5 (Hierarchical Data Format version 5). This format allows the storage of massive multi-dimensional datasets. In our system, the HDF5 file contains a 3D dataset where the dimensions are $(Time, X, Y)$.

Chapter 4

Implementation and Results

4.1 Development Environment and Technologies

4.2 Implementation of Hexagonal Architecture

4.2.1 Input

4.2.1.1 Multi-Format Raster File Input Adapter

Supporting IDRISI and HDF5 formats. (Aquí explicamos que un solo adaptador gestiona ambos).

4.2.2 Output

4.2.3 State Updater

4.3 Implementation of the Simulation Core

4.3.1 Algorithm for updating states using tensor operations

4.3.2 Performance optimization and parallelism

4.3.3 Tensor-based Grid Structure

4.3.4 The TemporalLayerManager: Efficient Memory Handling for Dynamic Data

(Aquí documentamos la lógica de actualización por pasos)

4.4 Data Extraction and Ingestion Process (Data Pipeline)

4.4.1 Elevation Layer

4.4.2 Water Depth Layer

4.4.3 Roughness Layer

4.4.4 Rainfall Layer

The implementation of the rainfall layer follows a decoupled pipeline where data acquisition and spatial normalization are performed offline using Python, while the real-time ingestion is managed by the C++ Core.

A. Pre-processing Pipeline (Python ETL)

Before the simulation begins, a series of Python scripts perform the **Extract, Transform, and Load (ETL)** process. This approach is chosen to leverage Python's extensive geospatial libraries (such as Pandas, Xarray, and Scipy).

1. **Extraction**: The scripts interface with the API or data repositories of the Hydrographic Confederations to retrieve raw meteorological time-series from physical sensors.

2. **Transformation (Spatial Interpolation)**: Since rainfall sensors are point-based, the scripts apply a spatial interpolation algorithm (e.g., Inverse Distance Weighting). To ensure consistency, the simulation's DEM is used as a spatial mask; the interpolation is clipped and resampled to match the exact grid dimensions and resolution of the elevation layer.

3. **Loading (HDF5)**: The resulting temporal sequence of rasters is serialized into a single HDF5 file. The data is organized into a 3D dataset (time, x, y), optimized for chunked access.

The following table details the scripts involved in the ETL (Extract, Transform, Load) process:

Script	Libraries	Core Responsibility
`rain_fetcher.py`	`requests, pandas`	Automated retrieval of sensor time-series from Hydrograph
`spatial_aligner.py`	`gdal, scipy`	Spatial interpolation of point-data and clipping to the DE
`hdf5_optimizer.py`	`h5py, numpy`	Serialization of the raster sequence into a chunked 3D HD

Table 4.1: Python scripts for Rainfall data pre-processing

B. Core Ingestion and Memory Management

In the implementation phase, the TemporalLayerManager class acts as the orchestrator for all time-varying data. This component ensures that the Core only allocates memory for the rainfall data that is currently relevant to the simulation time t.

Logic Workflow:

1. **Subscription**: During initialization, the rainfall layer is registered in the TemporalLayerManager with a specific update frequency (e.g., every 600 simulation seconds).
2. **Step Validation**: In each iteration of the Core, the manager checks if the current step requires a data refresh.
3. **Update Request**: If an update is needed, the manager calls the update method.
4. **HDF5 Access**: The manager uses the HighFive functions to perform a Hyperslab selection.
5. **Tensor Update**: The buffer is moved into the specific channel of the Core's grid tensor.

4.5 Testing and Validation

4.5.1 Test scenario: Simulation of a DANA event

4.5.2 Analysis of results and comparison with historical data or theoretical models

Pruebas, validación y los resultados de la simulación.

Chapter 5

Conclusions and Future Work

Resumen y líneas de investigación futuras.